HADES

AI Operating System

"The user should manage the goal — not the intelligence required to achieve it."

Human → Hades → AI Models / Tools / System → Verified Result

Functional Prototype · Linux-Targeted · Research & Engineering Demo

The Problem
Orchestration

Context Switching
Research on Gemini, coding on Claude, writing on ChatGPT — every task shift costs time, focus, and mental overhead.

Prompt Repetition
The same goals, constraints, and background must be re-explained to every model, every session, every time.

Lost Context
Notes in Notion, tasks in email, plans in chat — no persistent memory links intent across sessions or tools.

Workflow Management
No unified pipeline. The human manually sequences tools, transfers outputs, and rebuilds context at every handoff.

Model Selection
Which model for which task? The user must evaluate, choose, and re-route manually. There is no intelligent dispatcher.

Decision Fatigue
Models are powerful. The workflow is fragmented. The human IS the orchestration layer — and that is the core problem.

The Gap

CONVENTIONAL INTERACTION

Prompt → Response → Manual Orchestration

What is missing today:

- No explicit alignment gate before execution
- No mission-centric lifecycle management
- Conversation and execution are entangled
- AI self-declares success without verification
- Context is lost between sessions
- Model and tool selection falls on the user

The human becomes the orchestration layer — manually switching models, repeating context, managing workflow state, and deciding when a result is "good enough."

Models are powerful.

The workflow is fragmented.

HADES DESIGN THESIS

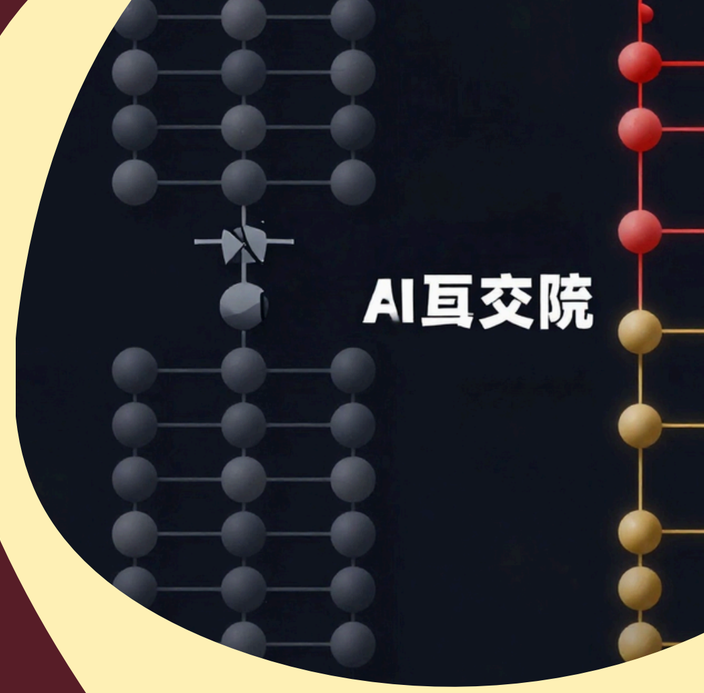
Intent → Mutual Understanding → Mission Lock → Coordinated Execution → Review → Memory

HADES architectural contributions:

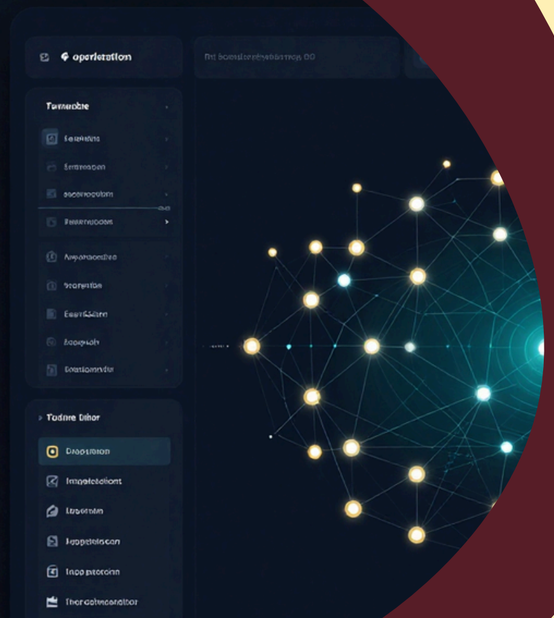
- Explicit alignment gate: execution is blocked until mutual understanding is confirmed
- Mission-centric lifecycle: every interaction anchors to a defined objective
- Separated conversation plane vs execution plane
- Verification before success claim: ReviewEngine validates outcomes deterministically
- Persistent context: memory survives across sessions
- Invisible orchestration: model and tool selection handled internally

The gap HADES targets is not model capability — it is the absence of a structured, verifiable, mission-aligned execution layer between human intent and AI action.

HADES design contribution: deterministic enforcement of alignment before execution.



What HADES Is



HADES IS:
✦ Persistent conversational partner — maintains context across sessions
✦ Mission-centric — every action tied to a declared objective
✦ Model-agnostic — routes to best AI regardless of vendor

HADES IS:
✦ Orchestration layer — coordinates models, tools, and system resources invisibly
✦ Long-term collaborator — builds shared context with the user over time

Core Philosophy:
"The human owns the goals and decisions. HADES owns the orchestration and execution intelligence required to achieve them."

HADES IS NOT:
✦ A chatbot — it does not just respond, it executes
✦ A prompt wrapper — architecture is structural, not cosmetic
✦ A one-shot command bot — it sustains missions over time

HADES IS NOT:
✦ A vendor-locked assistant — not tied to any single AI provider
✦ A blind executor — verification gates prevent unchecked action

Design Principle:
"Intelligence is infrastructure. The user manages the mission — HADES manages the intelligence required to achieve it."

Mission Lock Gate

FLOW: Intent → Clarification → Mission Extraction → Understanding Evaluation → ACKNOWLEDGE → AUTHORIZED_EXECUTION → Background Work. Conversation and reasoning are permitted; tool use and execution are fully blocked until lock is granted.

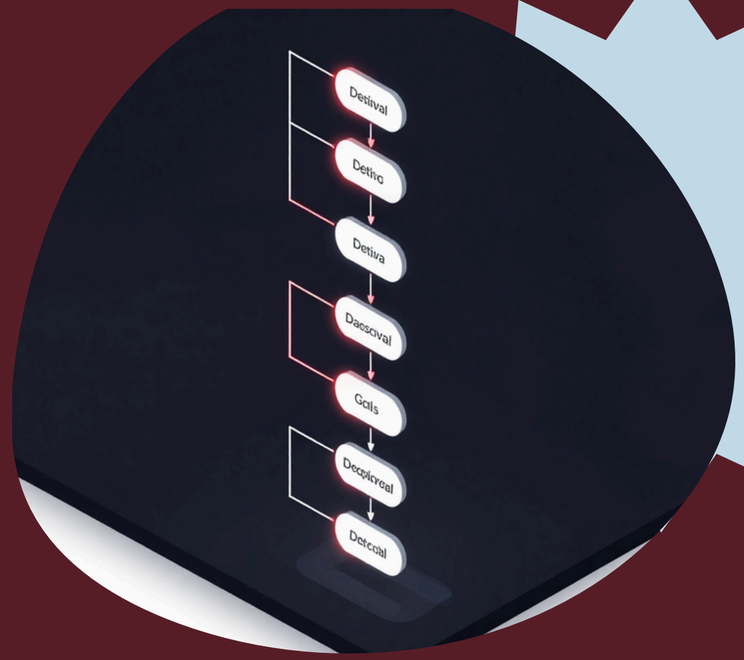
GATE FIELDS (all must pass): objective != empty; desired_outcome != empty; success_criteria != empty; mutual_understanding_reached = true. The system cannot proceed to execution until every field is satisfied deterministically.

PRE-LOCK RULE: While in conversation phase, the Partner Brain may ask questions, propose plans, and reason freely. No tool calls, no terminal commands, no file writes — zero side effects until ACKNOWLEDGE is issued.

DECISION OUTPUTS — three valid responses before lock: ASK (clarification needed, understanding incomplete); PROPOSE (plan offered, awaiting human approval); ACKNOWLEDGE (full alignment confirmed, execution authorized and mission locked).

FORCE_EXECUTE BYPASS: A developer-only override exists to skip the alignment gate for testing and rapid iteration. Clearly labeled as a testing tool only — not intended for production or end-user workflows.

WHY IT MATTERS: Most AI systems execute immediately on input. HADES enforces mutual understanding as a hard prerequisite. The mission lock is not a UX feature — it is a deterministic architectural gate separating conversation from execution.



System Architecture

CONVERSATION PLANE: User → Partner Brain → Mission Lock. Handles intent classification, clarification, mutual understanding gate, and memory retrieval. No tool execution permitted before ACKNOWLEDGE.

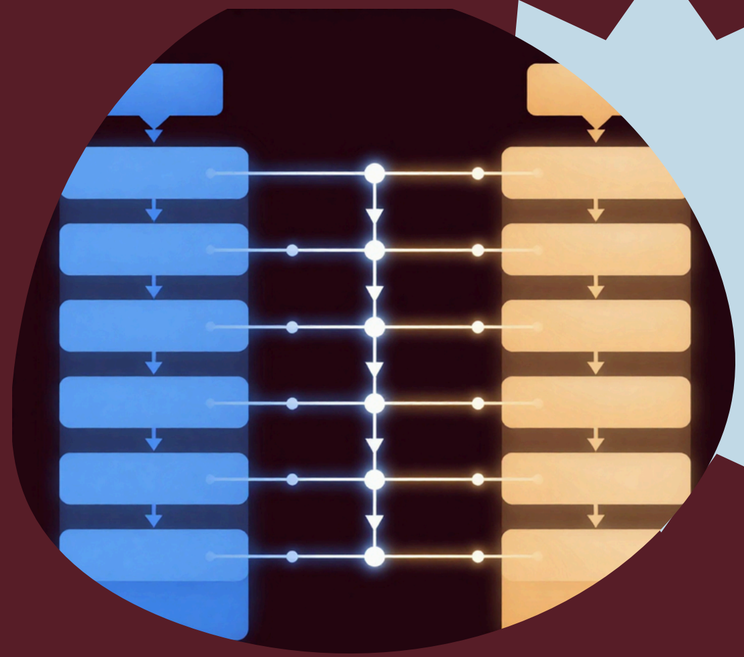
EXECUTION PLANE: Executive Brain → TaskGraph → WorkerManager / LiteLLM → Workers / LLMs → Skills & Tools → Host Linux Environment. Activated only post Mission Lock authorization.

VERIFICATION & DELIVERY: Review Engine (exit code + artifact + heuristic + semantic LLM check) → Memory Manager (JSON persistence) → SSE event stream → React/Vite/TypeScript/Tailwind Frontend + Kokoro-ONNX TTS.

BACKEND STACK: FastAPI · Python · asyncio · Pydantic schemas · LiteLLM unified gateway. Provider matrix: Gemini (conversational + analysis), Groq Llama (fast/open-source), OpenRouter (endpoint), Ollama (local/private).

SKILLS & TOOLS: TerminalSkill (shell exec), FilesystemSkill (read/write), ProcessSkill (daemon management). All sandboxed within Host Linux Environment. Tool dispatch routed by WorkerManager capability check + API-key availability.

MEMORY & STREAMING: Flat JSON memory store (append + retrieval). SSE push delivers MISSION_STARTED, TASK_UPDATE, MISSION_COMPLETED events in real time to frontend. Full async non-blocking pipeline via asyncio.create_task.



Two-Brain Architecture

PARTNER BRAIN

Intent Classification: Analyzes raw user input to determine request type and complexity scope.

Mission Extraction: Pulls structured objective, desired outcome, and success criteria from conversation.

Conversational Decision System: Outputs ASK / PROPOSE / ACKNOWLEDGE based on alignment state.

Memory Retrieval: Surfaces relevant past missions and context before execution begins.

Alignment Gate: Enforces mutual understanding before any execution is authorized.

EXECUTIVE BRAIN

Task Planning: Decomposes authorized missions into structured TaskGraph steps.

Delegation: Routes tasks to WorkerManager with capability requirements.

Tool Execution: Coordinates TerminalSkill, FilesystemSkill, ProcessSkill via Workers.

Retry / Recovery: Handles failures, reruns, and replan loops up to max_retries.

WORKERMANAGER: ROUTING LOGIC

Capability-Based Routing: Matches task requirements to worker skill profiles.

API-Key Availability Check: Validates provider credentials before assignment.

Priority Fallback: Degrades gracefully across provider tiers if primary is unavailable.

PROVIDER MATRIX

- Gemini — Conversational reasoning, fast response, primary partner brain.
- Gemini Pro — Deep research, analysis, complex multi-step planning.
- Groq Llama — Fast inference, open-source, low-latency execution tasks.
- OpenRouter — Flexible endpoint routing, model diversity fallback.
- Ollama — Local/private inference, offline capability, data-sensitive tasks.

WORKER SPECIALIZATION STATUS

⚠ PARTIAL: Current workers are model API endpoints, not autonomous specialist agents. Specialization is defined by model capability profile and routing rules, not independent agent logic. Full specialist worker architecture is a next-stage engineering target.

Review Engine

"Generation Is Not Success." AI cannot self-declare success — every mission output must pass deterministic verification before completion is claimed.

Stage 1 — Exit Code Check: Command execution must return exit code 0. Non-zero codes trigger immediate fail → retry/replan loop up to max_retries.

Stage 2 — Artifact Existence Check: Expected output file or resource must physically exist on the host filesystem. Missing artifact = failed mission, regardless of model confidence.

Stage 3 — Heuristic Validation: Output is inspected for non-empty content and expected structural patterns. Catches silent failures where files exist but contain garbage or empty data.

Stage 4 — Semantic LLM Review: Final layer checks semantic alignment with the original success_criteria. Only after all 4 stages pass does HADES emit MISSION_COMPLETED.

Deterministic Evidence Chain: exit code 0 → artifact exists → non-empty/expected patterns → semantic alignment. Fail at any stage → retry loop. No self-reported success allowed.

Complete Mission

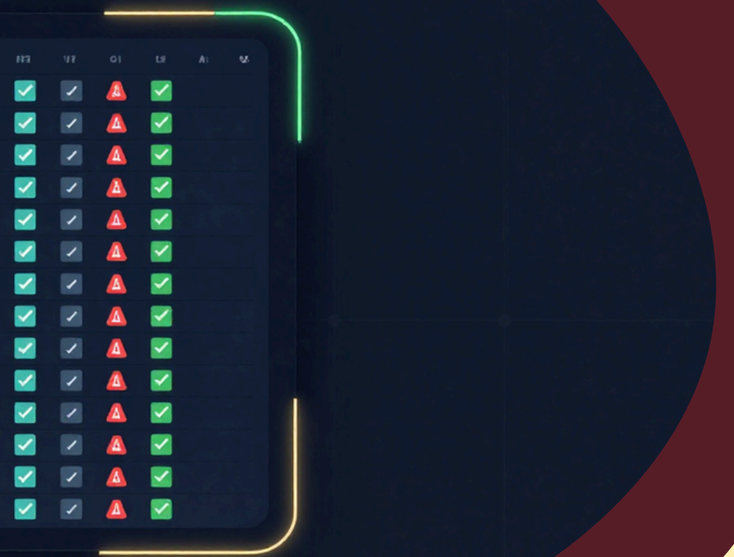
IMPLEMENTED END-TO-END

Mission: "List all files in the current directory and save them to output.txt."

1. POST /api/chat — user message received by FastAPI backend
2. Classify SMALL_TASK — Partner Brain intent classifier fires
3. Extract mission — objective, outcome, success_criteria parsed
4. Evaluator authorizes — mutual_understanding_reached = true; ACKNOWLEDGE issued
5. asyncio.create_task — non-blocking execution task spawned
6. Single-task TaskGraph — Executive Brain builds one-node graph
7. WorkerManager routes Gemini — capability check + API key verified
8. Gemini generates command — produces: ls -la > output.txt
9. TerminalSkill executes — subprocess runs in host Linux environment
10. Exit code 0 — deterministic success signal captured
11. ReviewEngine checks output.txt — artifact existence + heuristic + semantic LLM review pass
12. memory.json append — mission record persisted to flat JSON store
13. SSE MISSION_COMPLETED — event streamed to frontend; UI updates + Kokoro TTS speaks result

Every step above is implemented in the current prototype codebase. No simulated or mocked stages. This is the live execution path for a real terminal mission from intent to verified delivery.

Current-State Audit



✅ IMPLEMENTED
Conversation Alignment · Mission Lock · State Machine · Model Routing · Terminal / Filesystem / Process Tools · SSE Streaming · Backend TTS · E2E Execution Path

⚠️ PARTIAL
Task Decomposition (hardcoded 1-task planner) · Worker Specialization (model endpoints, not specialist agents) · Long-Term Memory (flat JSON, no RAG)

❌ MISSING
Backend STT · Sandboxing / Isolation · Structured Logging · Comprehensive Test Suite · Dynamic DAG Planner · Vector Memory Retrieval

IMPLEMENTATION STATUS LEGEND
✅ WORKING end-to-end in current prototype build
⚠️ PARTIAL — scaffolding exists, not complete
❌ MISSING — not yet implemented or planned next

MATURITY SNAPSHOT — Engineering Estimates
Core 60% · Frontend 75% · Backend 65% · AI/LLM 70% · Mission System 80%
Memory 20% · Workers 50% · Tools 40% · Review 60% · Voice 30%
Automation 20% · Security 5% · Testing 10% · Documentation 80%

HONEST ASSESSMENT
"These are engineering-audit estimates, not benchmark metrics. The core execution loop is proven. Significant work remains in security, memory, and testing before production readiness."

What Is Genuinely Novel

Mutual Understanding Gate
Mechanism: Deterministic pre-execution gate enforces objective, desired_outcome, success_criteria != empty and mutual_understanding_reached = true before any tool or execution call is permitted. Not a UX feature — a hard architectural constraint.

Mission-Centric Computing
Mechanism: Interaction is structured around a persistent mission lifecycle — Intent → Lock → Execution → Review → Memory — not a stateless prompt/response loop. The mission is the first-class system primitive, not the message.

Model-Agnostic Routing
Mechanism: WorkerManager performs capability-based provider selection with priority fallback. No vendor lock-in at the architecture level. Local (Ollama) and cloud (Gemini/Groq/OpenRouter) endpoints are interchangeable execution targets.

Invisible Intelligence Orchestration
Mechanism: Model and tool selection is fully abstracted from the user. HADES routes to the best available provider (Gemini, Groq, Ollama, OpenRouter) based on capability and API-key availability — the user never selects a model.

Partner / Executive Brain Duality
Mechanism: Conversation and execution are handled by structurally separated cognitive planes. Partner Brain owns alignment and intent; Executive Brain owns task planning and delegation. Cross-plane tool use is blocked until mission lock is achieved.

Verification-First Execution + Persistent Relationship
Mechanism: ReviewEngine applies 4-stage pipeline (exit code, artifact, heuristic, semantic LLM) before declaring success — AI cannot self-certify. Combined with append-only memory, HADES treats the ongoing relationship as a system primitive, not a session.

Gaps → Roadmap

🔴 Host Security Risk → Docker/Sandbox Isolation. Current direct Linux execution is unsafe for untrusted commands. Target: containerized execution layer with resource limits and escape prevention.

🔴 Hardcoded 1-Task Planner → Dynamic DAG/TaskGraph Decomposition. Current TaskGraph handles only single-step missions. Target: dynamic multi-step dependency graph with parallel execution and replan-on-failure.

🟡 Flat JSON Memory → Vector/RAG Semantic Retrieval. Current memory is append-only flat JSON with no semantic search. Target: embedded vector store with RAG retrieval for context-aware long-term memory.

🟡 Browser-Only STT → Backend Whisper / Offline STT. Voice input currently depends on browser Web Speech API. Target: backend Whisper integration for offline, privacy-preserving, robust speech-to-text.

🔴 Thin Tests & Observability → E2E Test Suite + Structured Logging. No integration tests; logging is unstructured. Target: pytest E2E suite, structured JSON logs, OpenTelemetry metrics, and mission replay tooling.

▶ Maturity Arrow: Prototype → Orchestrated Agent → Secure Execution System → AI Operating Layer. Next-stage trajectory: systemd/D-Bus Linux integration, recurring/event-driven missions, specialist workers, multi-device continuity.

HUMAN SETS THE MISSION.
HADES ORCHESTRATES THE INTELLIGENCE.

Human → Hades → Models / Tools / System → Review → Memory → Human

Proof · Conversation Gate · Model Routing · Verified Execution · Real-Time Delivery

Today: functioning AI execution prototype. Next: secure multi-step, memory-rich, autonomous AI operating layer.